



**HAWASSA UNIVERSITY**  
**INSTITUTE OF TECHNOLOGY**  
**FACULTY OF ELECTRICAL ENGINEERING**  
**DEPARTMENT OF ELECTRICAL AND COMPUTER**  
**ENGINEERING**  
**(COMPUTER STREAM)**  
**THESIS PROJECT**  
**TITLE: GATE PASS MANAGEMENT SYSTEM**

**Group members:-**

| <b><u>No.</u></b> | <b><u>Name</u></b> | <b><u>ID</u></b> |
|-------------------|--------------------|------------------|
| 1.                | BINYAM YOHANNES    | Tech/1397/09     |
| 2.                | DAWIT DESALEGN     | Tech/0522/10     |
| 3.                | ENDASHAW NIGATU    | Tech/0621/10     |

Advisor Name: \_\_\_\_\_ Signature \_\_\_\_\_

11/06/22

### **Declaration**

We, the undersigned, declare that this Project is our original work, has not been presented for a degree in this or any other universities, and all sources of materials used for the Project have been fully acknowledged.

| Name               | Signature |
|--------------------|-----------|
| 1. Binyam Yohannes | _____     |
| 2. Dawit Desalegn  | _____     |
| 3. Endashaw Nigatu | _____     |

Date of submission:

This project has been submitted for examination with my approval as a university advisor.

| Project advisor | Signature | Date  |
|-----------------|-----------|-------|
| _____           | _____     | _____ |

## **Acknowledgment**

First we want to thank God for everything. We would also like to express our gratitude to our Advisor Mr. Muluneh for the continuous support of the project, for his patience, motivation, enthusiasm and immense knowledge. His guidance helped us in all the time of the project. We thank our friends in the electrical and computer engineering department for the stimulating discussions, for the sleepless nights that we were working together, and for all the fun we had in the last five years. Most importantly, none of this could have happened without our family. We also place on record our sense of gratitude to everyone whom directly or indirectly have let their hand in this project.

## Table of Contents

|                                           |      |
|-------------------------------------------|------|
| Declaration                               | ii   |
| Acknowledgment                            | iii  |
| Table of Contents                         | iv   |
| List of Figures                           | vi   |
| List of Tables                            | vii  |
| List of Acronyms                          | viii |
| Abstract                                  | ix   |
| CHAPTER ONE                               | 1    |
| INTRODUCTION                              | 1    |
| 1.1 Background of the Study               | 1    |
| 1.2 Statement of the Problem              | 2    |
| 1.3 Objectives                            | 2    |
| 1.3.1 General Objective                   | 2    |
| 1.3.2 Specific Objectives                 | 3    |
| 1.4 Scope                                 | 3    |
| 1.5 Significance Of The Project           | 4    |
| 1.6 Limitation Of The Project             | 5    |
| CHAPTER TWO                               | 6    |
| LITERATURE REVIEW                         | 6    |
| CHAPTER THREE                             | 9    |
| SYSTEM ANALYSIS AND DESIGN                | 9    |
| 3.1 System Analysis                       | 9    |
| 3.1.1 Methodology                         | 9    |
| 3.1.2 Functional Requirements             | 10   |
| 3.1.3 Non Functional Requirements         | 11   |
| 3.1.4 Tools                               | 13   |
| 3.1.5 Building android mobile application | 14   |
| 3.2 System Design                         | 15   |
| 3.2.1 Database Design E-R Diagram         | 15   |
| 3.2.2 Database Table                      | 16   |
| 3.2.3 Use Case Diagram                    | 17   |

|                             |    |
|-----------------------------|----|
| 3.2.4 Use Case Descriptions | 17 |
| 3.2.5 Class Diagram         | 20 |
| 3.2.6 Sequence Diagram      | 21 |
| CHAPTER FOUR                | 24 |
| RESULT AND DISCUSSION       | 24 |
| CHAPTER FIVE                | 32 |
| 5.1 Conclusion              | 32 |
| 5.2 Recommendation          | 32 |
| Appendix                    | 34 |

## List of Figures

|                                                             |    |
|-------------------------------------------------------------|----|
| Figure 1 Waterfall model                                    | 9  |
| Figure 2 ER diagram                                         | 15 |
| Figure 3 Database table                                     | 16 |
| Figure 4 Use Case Diagram                                   | 17 |
| Figure 5 Class diagram                                      | 21 |
| Figure 6: sequence diagram for admin                        | 22 |
| Figure 7 Sequence diagram for HRM generate id               | 22 |
| Figure 8: Sequence diagram for Guard add visitor            | 23 |
| Figure 9: Sequence diagram for Inviter (HO) invite visitors | 23 |
| Figure 10: Login Page                                       | 25 |
| Figure 11: Guard homepage                                   | 26 |
| Figure 12: Waiting to scan the barcode                      | 26 |
| Figure 13: barcode when an employee enters                  | 27 |
| Figure 14: barcode reader when it is not employee           | 27 |
| Figure 15: guard adds visitor                               | 28 |
| Figure 16: Guard request view page                          | 28 |
| Figure 17: HRM report homepage                              | 29 |
| Figure 18: HO homepage                                      | 29 |
| Figure 19: HO creates invitation                            | 30 |
| Figure 20: Admin homepage                                   | 30 |
| Figure 21: scanning using app                               | 31 |
| Figure 22: result of scanning the barcode                   | 31 |

## List of Tables

|                                                     |    |
|-----------------------------------------------------|----|
| Table 1: use case description for manage account    | 18 |
| Table 2: use case description for login             | 18 |
| Table 3: use case description for view report       | 19 |
| Table 4: use case description for view guard detail | 19 |
| Table 5: use case description for restrict entry    | 19 |
| Table 6: use case description for entry gate detail | 19 |
| Table 7: use case description for Add visitor       | 20 |
| Table 8: use case description for invite visitor    | 20 |

## List of Acronyms

|       |                                      |
|-------|--------------------------------------|
| ICT   | Information Communication Technology |
| PhD   | Doctor of Philosophy                 |
| MA/MS | Master of Arts/Master of Science     |
| FTI   | Further Training Institute           |
| GTP   | Growth and Transformation Plan       |
| ER    | Entity relationship                  |
| ICT   | Information communication technology |
| PHP   | Hypertext pre processor              |
| SRS   | Software requirement specification   |
| GPMS  | Gate Pass Management System          |
| HRM   | Human Resource Manager               |
| HO    | Higher Official                      |



### **Abstract**

“Digital is way better than the paperwork”, the main concept of the system is to enhance the security process while reducing the involvement of human resources. Enhancing security at the gate level is essential in every organization. Poor security at gate level to track existing materials and its movement, transit of employees during the working hours and visitors entering the premises could affect the reputation of the whole organization. The Gate Pass is an important document for this purpose. It will be great to implement an automated process for Gate Pass Management that aims to replace the manual process. The manual system results in a lot of inefficiency and unaccountability due to cumbersome and inaccurate record keeping with inconsistent maintenance of security about the movements. The methodology used for the development and implementation of the system is “Waterfall Model”. The generic approach for software designing is Top-Down design. Also, PHP, MySQL, and WAMP were utilized as software in system development and implementation.

## CHAPTER ONE

### INTRODUCTION

#### 1.1 Background of the Study

Most Vulnerable point in an organization is the front door. Aside from physical barriers like doors and locks, organizational information has become today's single-most critical security mechanism.

In all organizations, irrespective of their capacity, a large number of movements take place every day at the gate level. It may be employees moving in or out, visitors entering the premises, movement of materials etc. All these movements occur at the organization's premises, factories and companies need to be monitored and controlled. This is where the concept of a gate pass comes into play. Gate pass can be used to authorize the movements of humans, materials and machines to or from the premises of the organization. It will help to monitor and track all the movements happening in an organization.

Traditionally, manual record management required vast amounts of documents to be shipped to storage facilities only to necessitate retrieval when needed and has resulted in the unnecessary expense of both time and money. On the other hand, many innovations are now being developed to fasten its transaction. One of the innovations is the E-Gate Pass System, which aims to enhance and upgrade the existing system by increasing its efficiency and effectiveness by reducing manual work. Further, the software improves the working methods by replacing the current manual system with the computer-based system. As safety and security is a concern, it is stressed that the consensus arising from the professional security community is that university administrators should invest in sophisticated technologies that help university staff to decrease violence via a multi-staged approach to safety. A traditional-manual process may lead to inconsistency of information and difficulty in generating records.

Automated Gate Pass Management System aimed to modernize the manual pass slip system, which will be considered a technology to address the gap using PHP, WAMP, and MySQL. Thus, this study is aimed to address the present situation in the Gate Pass Management Present System, designed to manage records, particularly in facilitating information for an employee,

visitors and resources they take and gate through the gate. Records are accessed to an informed and effective decision-making process of the administration.

A gate pass can be used to authorize the movement of people and items into and out of an organization's grounds. It will aid in the monitoring and tracking of all activities taking place within an organization. Enhancing security at the gate level is critical in every organization, since poor security of current items and their movement, employee transit during working hours, and guests accessing the grounds could jeopardize the entire firm's reputation.

When a gate pass system is deployed in a company, it can provide numerous benefits. The first is the restriction of unlawful movement within an organization's premises; aside from that, the gate pass allows the organization to keep track of the time of movement and the person responsible for it. When it comes to materials, a gate pass keeps unlawful transactions of materials. The system enhances data accuracy and makes it easier to get data out of the system. The system cares for the overall security of the organization while improving the discipline inside the organization. It also enhances the company image by incorporating systems that are technologically sound. This gives a positive view for people who visit the organization. Once they experience the methods that have been used by the company will increase their positive attitudes towards the organization.

## **1.2 Statement of the Problem**

Enhancing security at the gate level is essential in every organization, as the poor security of material existing and its movement, transit of employees during the working hours and visitors entering the premises could ruin the image of the whole organization.

Regarding the register does not mean that information is true. There is no authentication process. With lack of a digital solution, information has to be entered manually and thus making it easy for data loss.

## **1.3 Objectives**

### **1.3.1 General Objective**

To develop an Automated Gate Pass System designed to monitor and facilitate the process and information of the students, employees, and visitors passing campus gate.

### 1.3.2 Specific Objectives

- To improve the efficiency and accountability of the security process by using an automated system.
- To improve the data management by making the link with respective departments and security desk.
- To establish a system which will align with other automated systems to increase the traceability and compliance of records, to the requirements of the organization.
- To design and develop a system with the following features and modules:
  - a. Gate pass module
  - b. End-user module
  - c. Admin module
  - d. Reports module
  - e. Registration module
- To integrate an Automated Gate Pass System supported with PHP, WAMP and MySQL
- To build android application for security guard to scan barcode given for the employees
- To evaluate the system in terms of usability and functionality.

### 1.4 Scope

The introducing system, gate pass management system, which is going to be implemented for organizations, will automate the operation of the gate pass system of the center.

This web-based gate pass management system allows users (HR, HO, guard and administrator) to get registered on the web platform and gives the access to write gate details to the administrator. It is supported with a well-designed database. A friendly user interface is provided to facilitate different services such as login, generate id, submit and view reports. Although this system is designed for gate pass security for employees, visitors and resources they take, we can make it available to all entry and exit since it is flexible. Achieving a secure environment is very important for all organizations as it is a matter of security. Our application would help such universities to safeguard their compound from unwanted personnel and it is also safe for different kinds of items.

The application should accomplish the following:

- Issue Gate passes to employees within an organization.
- Issue Gate passes to visitors within an organization.
- Issue Gate passes to resources taken by employees and visitors within an organization.
- This study focuses as stated, on the security of the organization. Here the material, employees and visitors considered, and this automated system follows a unique procedure to help maximize the security of above resources.

### 1.5 Significance Of The Project

The security of an organization depends on a lot of factors and if it is less attentive, then might risk the security of the entire organization. To make sure that everything is under control, must implement the right measures and this drives the organization towards a solution that takes care of the security at the gate. Gate Pass Management System enables the right security, where one could keep a vigilant eye on the material movement, transit of employees during working hours and the visitors who gain the access to the premises can gain numerous advantages when gate pass system is implemented in an organization. The first and foremost being the restriction of unauthorized movement in organizations' premises, and apart from this, the gate pass allows the organization to get a record of the time of movement and to track the person responsible for the movement. In the case of materials, gate pass prevents unauthorized transactions. The system improves the accuracy of data and eases the process of fetching data out of the system. The system cares for the overall security of the organization while improving the discipline inside the organization. It also enhances the company image by incorporating systems that are technologically sound. This gives a positive view for people who visit the organization. Once they experience the methods that have been used by the company will increase their positive attitudes towards the organization. "Digital is way better than the paperwork", the main concept of the system is to enhance the security process while reducing the involvement of human resources. Generally, the Gate pass management system keeps a check on visitors at the entry and exit points. Unlike a manual system, Gate Pass Software is time-efficient; it provides approval only after checking all the records in the database. The records are accurately maintained by the management system.

### **1.6 Limitation Of The Project**

Even Though gate pass management systems do an effective job regarding security and gate control it has some limitations. One of the main limitations of this project is the waiting line of users that wanted to use the system at the same time. The system can handle a specific number of people depending on the technology and the devices that the organization provides at the gate area. If many people start to interact with the system at the same time more than it can handle, there will be a waiting line. And these issues can be handled by deploying large amounts of human resources and also having a number of the latest technologies available.

## CHAPTER TWO

### LITERATURE REVIEW

Touch Point Visitor Management is futuristic web-based software to manage visitors at the office. It does away with the obsolete pen and paper system of gathering visitor information and brings in a professional approach to visitor management. No more scribbling in the visitor book, issuing used badges, calling the employee to inform of the visitor's arrival, unattended visitors in crowded reception areas and most importantly, worrying about security issues. Manual systems are boring and time consuming. This visitor management software records all-relevant information about the visitor, which is automatically captured in a database. Then ID scanning is done, and a professional quality visitor badge/pass is printed. [1]

This web-based system covers only the transaction of the visitors. The additional feature that is available is the ability to print a professional visitor badge/pass by scanning visitor's ID. Since the scanning of different types of visitor's ID is a considerable process at the gate level, the newly implementing system will only print the generated gate pass at the security desk. [1]

A gate pass as the name suggests is the activity of the security personnel and is the focal point of an organization's security set up. The gate pass is an essential document to monitor the Material movement in and out of an organization. Topnotch provides a unique module, which assists you in professionalizing the manner in which you control the Material flow of which the gate pass is an integral part. Materials play a vital role in the profitability of your business organization. Topnotch is your answer to increasing cost effectiveness by satisfying your material tracking needs. And increase the efficiency, productivity, and customer satisfaction of your businesses.

- Record entry, exit of Visitors, Contract workers, Vehicles and Materials
- Authorize all entries & Allow, Forward or Reject a request
- Pass formats: Use from the database or design your own
- Detailed onscreen and back-office MIS & One day and long-duration, multi-entry passes  
Custom data entry forms (fields and labels) Informative control centers – current visitors, visit history, waiting for authorization, online users
- Data Import: Users, departments, contract workers
- Intuitive and simple user interface for security users

- User and Role definition[2]

This system covers the gate pass process of materials, visitors and vehicles. The main difference between this system and the newly implementing system is that it covers the vehicle gate pass process also. But absence of a gate pass process for employee transactions is the main disadvantage of this system. Additional features available are multi-entry passes. Even though the above-mentioned system covers the transaction of company vehicles, I haven't considered this as a requirement in the newly implementing Gate Pass system. Multi – entry passes were disallowed in the newly implementing system to ease track of the data entries. [2]

The WINHMS Gate Pass module monitors and maintains gate registers for materials, visitors and company vehicles. Returnable and non-returnable pass types are supported for both inward and outward movements. In the case of returnable gate passes, the original gate pass transaction is always referenced so that the pending returnable details can be obtained from the system at any point of time. Materials dispatched may be from user departments or material stores. When materials are routed through a store, store issue documentation is referenced during gate pass creation. The system automatically generates unique gate pass sequence numbers individually for inward and outward gate passes. Movement of vendors, customers, sales personnel and visitors are maintained and tracked at all times. Within the visitor input form, the visitor's name, address, company details, purpose of visit, designated contact etc. are captured. Reports are provided for registers such as material inward register, pending gate pass register, vehicle movement register and vehicle availability status. This system covers the gate pass process of materials, visitors and company vehicles.[3]

The main difference between this system and the newly implementing system is that it covers the company vehicle gate pass process also. But absence of a gate pass process for employee transactions is the main disadvantage of this system. Even though the above-mentioned system covers the transaction of company vehicles, I haven't considered this as a requirement for the newly implementing Gate Pass system. [3]

The Gate-Pass Management system by applying the model of UTAUT was discussed by Norizan Anwar, Mohamad Noorman Masrek, Yanty Rahayu Rambli in 2012. They proposed the system by adopting a technology model (UTAUT) to determine the user acceptance of visitor



application systems. The main motto of this system was Handling your visitor at your fingertips.[4]

Another system for Visitor Pass was discussed in the paper by Prof. Abhay Gaidhani, Suraj Sahijwani, Parag Jain, Shantanu Jadhav, Ankush Jain in 2015. This paper aims to develop a system for Gate pass using Raspberry Pi. The main aim was to save paper with the help of Internet Connectivity to send SMS and Email for verification of users. [5]

Digital Visitor Information Management System (VIMS) Application and Design. This application enables capturing new visiting records by auto-clock in/out, and assignment of visitor pass. Visitor information is recorded in a centralized database server, which provides data management and manipulation through searching and report generating. E-VIMS able to record visitor information during visitor registration by using visitors Malaysia Government Multipurpose Card (MyKad).[6]

On the other hand, the decision to develop a visitor system may depend on the needs of an organization so in this review paper we have discussed a Visitor Gate Pass Management System to enhance level of security and manage visitor in any Organization.[7]

## CHAPTER THREE

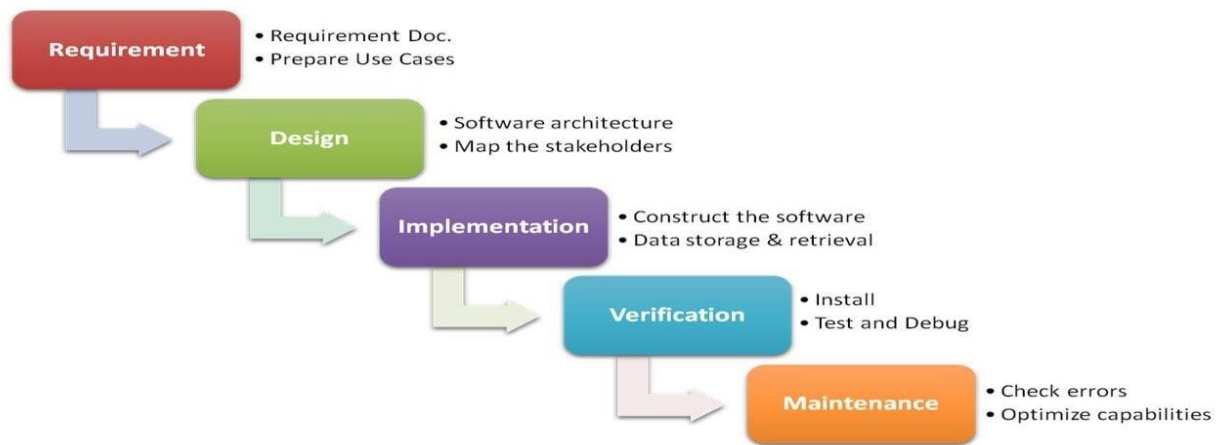
### SYSTEM ANALYSIS AND DESIGN

#### 3.1 System Analysis

System analysis is the act, process or profession of studying an activity in order to define its goal and purposes and to discover operations and procedures for accomplishing them most efficiently. It is a process of collecting and interpreting facts, identifying the problems, and decomposition of a system into its components.

##### 3.1.1 Methodology

We will use the Iterative and Incremental Development model (IID) for our project development. This development approach is also referred to as the Iterative Waterfall Development approach.



*Figure 1 Waterfall model*

Methodology specifies the method and technology used to develop the software system such as, the methods used to gather data, approach used to design the software system, software and hardware requirements used to implement the system. We have used the following phases: -

#### A. Data Gathering Method

- Document analysis: - we will use existing files in different sectors like visitor and material gate pass management systems in order to gather information on how we should interact with their mission, goal, and service so that we will make our system suitable to interact with them.
- Interview: - stratified random sample method will be used which we will interview from selected strata to get common ideas about gate pass system requirement. These data gathering sessions will also be done repeatedly.

Observation: - which will help us to personally understand what is going on in this area of work and give our professional solution to some cases.

### **B. Analysis, Design and Architecture**

Since there will be continuous change we choose to use the waterfall methodology.

For the system's website, client server architecture style will be used.

Model-View-Control (MVC) architectural pattern will be used since it enables all of the developers to work simultaneously on the model, controller and view and also enables low coupling between components.

#### **3.1.2 Functional Requirements**

Detailed description of what the system can do as well as input and Output of the System with respect to the role of the actor. The product consists of the following functional requirements based on the role of the actors.

- Admin: is basically a super user.
  - Can add a record of HRM
  - Can add a record of Guard.
  - Can add a record of HO (Higher Official).
  - Responsible for any error in the system.
  - Admin should keep tracking each person and different resources entering and exiting the compound .
- HRM (Human Resources Manager): is part of an organization who is dealing with each guard.

- The HRM needs to keep track of guards and areas under his supervision.
- The manager can view entry gate details, Guard details.
- They generate employee id.
- Guard (security officer): Guard is the basic unit of the Gate Pass Management system.
  - Guard can view entry gate details,
  - They can accept or reject the entry of a person based on a barcode for employees and invitation list for visitors. They would allow only if the gate pass is approved.
  - They will see any unwanted activity restrict the entry of a person or related resources.
- HO (INVITOR)
  - Can add a record of visitors.
  - Prepare invitations.

### 3.1.3 Non Functional Requirements

Non-functional requirements are the criteria used to judge the system rather than the system's behavior. They include:

#### I. Performance Requirement

The system must be responsive, quick to reply to requests. Accuracy on data filling and saving is must. The system must be fast to give searching results. Quick help giving section must be included in the system for better understanding of users.

#### II. Safety Requirement

Data loss is intolerable so backup is must once in a while. Neither is error while saving information therefore there must exist some way of validation before saving the information.

#### III Security Requirement

- The backend will be developed using Laravel which is a secure framework these days. It provides:
  - **Protection against SQL injection in Laravel:** SQL Injection (SQLi) is a hacking technique where malicious SQL statements are inserted into an entry field and executed. This gives attackers control over the database. They can modify, disclose or delete the data — up to and including wiping the entire database.

Laravel uses the Eloquent ORM (object relational mapper) that does not allow malicious query data to pass through the forms. Due to PDO parameter binding, Eloquent ORM escapes these SQL commands and saves the invalid queries as text. This means our system will be protected from SQL injections as long as Laravel uses Eloquent ORM or Fluent Query Builder. This significantly improves security.

- **Protection against cross-site request forgery in Laravel:** attackers use cross-site request forgery (CSRF) to perform unwanted actions on the part of authenticated users. Malicious requests are sent to the target site from another site that the user visits. The target website believes the forged requests to be legitimate ones coming from the logged-in user. Laravel has CSRF protection enabled out-of-box. For this, it uses CSRF tokens. Tokens are generated on the form entry and compared against the ones saved in the user session. If there is no match, the request is not executed.
- **Protection against cross-site scripting in Laravel:** Cross-site scripting (XSS) allows attackers to inject malicious scripts into the content of trustworthy websites. These scripts travel with dynamic content to the user's browser and are executed there. In this way, attackers take advantage of vulnerabilities in a website that a user visits. Laravel prevents cross-site scripting, because its syntax helps to automatically escape HTML objects that are passed via a view variable. XSS filter lets you remove the html tag from input value.
- **Password protection in Laravel:** Unlike other forms of encryption, hashes are not designed to be decrypted with any keys. This is a "one-way" action. When a user enters a password again, its hash is verified against the one received previously.[4]

#### IV. Interoperability

The system can be used in any standard browser, using any operating system.

#### V. Software Quality Attributes

##### Usability

The system will not be dependent on any processor so will be usable. And also will have a user friendly interface so that any user can use it easily.

**Reliability**

The system can be relied upon to do what it is expected since its underlying architecture is well-built.

**Maintainability**

The system is coded in PHP and MySQL for the backend and HTML5, CSS for the frontend. These programming and markup languages are easy to understand hence ensuring that anyone competent enough can modify the system accordingly. This is further supported by comments in the code explaining how complex parts of the code work.

MVC (Model, View and Controller)

MVC separates an application into three components - Model, View, and Controller.

**Model:** Model represents the shape of the data. A class in PHP is used to describe a model. Model objects store data retrieved from the database. Model represents the data.

**View:** View in MVC is a user interface. View displays model data to the user and also enables them to modify them. View of MVC is HTML, CSS, and some special syntax (Razor syntax) that makes it easy to communicate with the model and the controller. View is the User Interface.

**Controller:** The controller handles the user request. Typically, the user uses the view and raises an HTTP request, which will be handled by the controller. The controller processes the request and returns the appropriate view as a response. Controller is the request handler.

**3.1.4 Tools****Hardware Tools**

- Personal computer (PC) or laptop: almost all tasks of our project are performed on a computer.
- Flash: required for data movement.
- Ethernet cable: to connect with the internet.
- Barcode reader: to read employee/visitor id

- Android phone

#### **Software Tools**

- Laravel: PHP framework
- Josh: Laravel template
- Wamp Web server
- Chrome: Web browser
- VS code: Text editor
- Wonder share EdrawMax: Diagram builder
- Flutter framework

#### **3.1.5 Building android mobile application**

An android app is a software application running on the android platform .because the android platform is built for mobile devices.

For our project we built an android app for security guard .which helps the security guard to scan the employee id and the visitor id at the gate and verifying their id using post method and gate method API from the website database in order to check the legality of them.

We used to build the android application flutter framework. Flutter is Google's free and open-source UI framework for creating native mobile applications .Flutter allows developers to build mobile applications with a single codebase and programming language. This capability makes building both iOS and android apps simpler and faster.

Building mobile applications with a flutter framework will do so using a programming language called Dart with a syntax like JavaScript .Dart is a typed object-oriented programming language that focuses on front end development.

#### **Why we choose flutter framework**

- Increased productivity .Using the same codebase for iOS and android saves both time and resources.
- Easy to learn
- Great performance

- Cost -effective
- Available on different IDEs. It is free to choose between android studio and vs code to edit their code on flutter.

## 3.2 System Design

In the previous chapter we have seen different steps, approaches, main processes or requirements which are involved in the gate pass management system. In this chapter, we will analyze the requirements of the system in depth in order to identify its key components to create a system that will achieve its goals in an efficient way. We will try to explain the stated requirement with some technical representation. The main part of the chapter covers requirement modeling based on object-oriented modeling techniques which is the UML (Unified Modeling Language). We will use models; Use-case diagrams, Sequence diagrams and Class diagrams to conceptualize and construct the proposed system.

### 3.2.1 Database Design E-R Diagram

A basic ER model is composed of entity types and specifies relationships that can exist between entities.

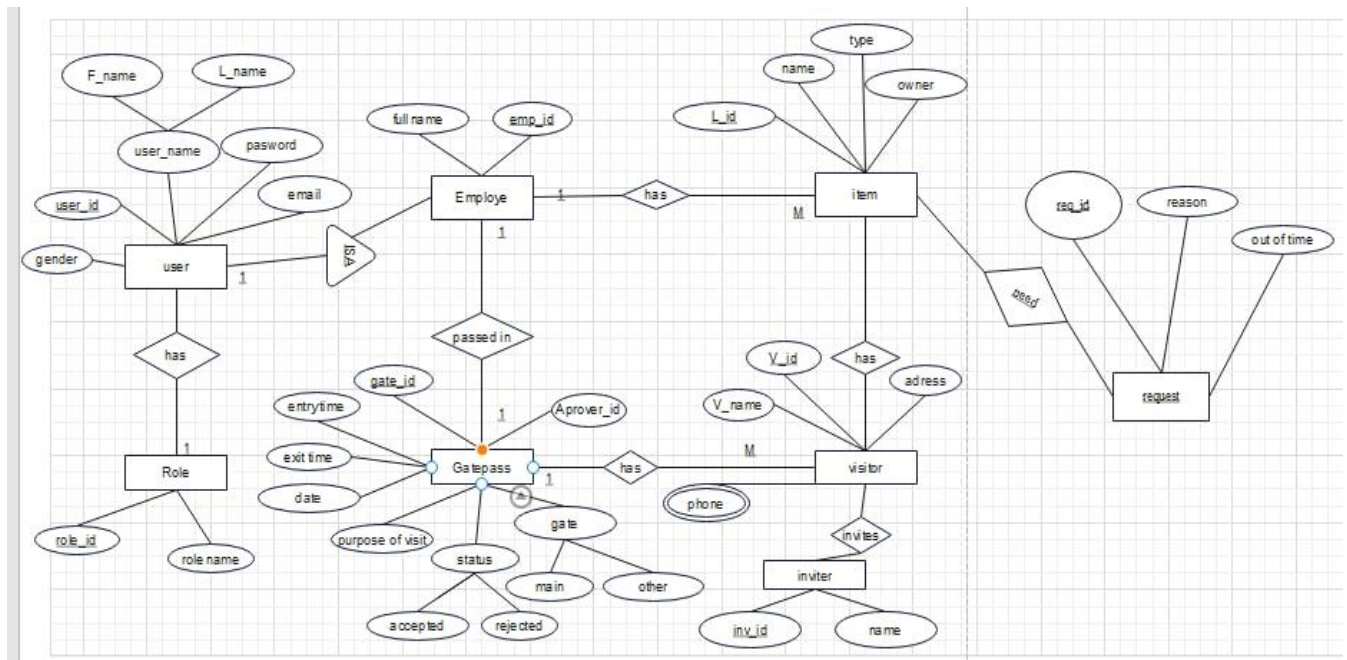


Figure 2 ER diagram

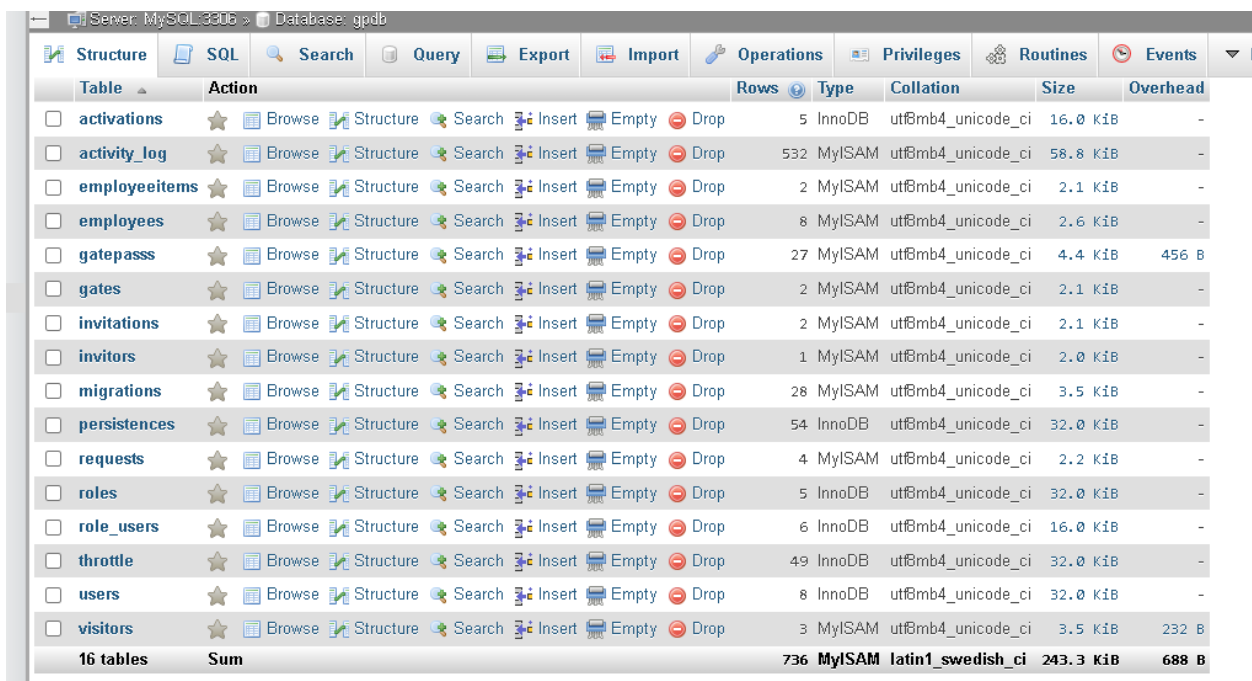


### 3.2.2 Database Table

A table is a collection of related data held in a table format within a database. It consists of columns and rows.

In relational databases, and flat file databases, a table is a set of data elements (values) using a model of vertical columns (identifiable by name) and horizontal rows, the cell being the unit where a row and column intersect. A table has a specified number of columns, but can have any number of rows. Each row is identified by one or more values appearing in a particular column subset. A specific choice of columns which uniquely identify rows is called the primary key.

"Table" is another term for "relation"; although there is the difference in that a table is usually a multisite (bag) of rows where a relation is a set and does not allow duplicates. Besides the actual data rows, tables generally have associated with them some metadata, such as constraints on the table or on the values within a particular column.



| Table            | Action                                      | Rows       | Type          | Collation                | Size             | Overhead     |
|------------------|---------------------------------------------|------------|---------------|--------------------------|------------------|--------------|
| activations      | ★ Browse Structure Search Insert Empty Drop | 5          | InnoDB        | utf8mb4_unicode_ci       | 16.0 KiB         | -            |
| activity_log     | ★ Browse Structure Search Insert Empty Drop | 532        | MyISAM        | utf8mb4_unicode_ci       | 58.8 KiB         | -            |
| employeeitems    | ★ Browse Structure Search Insert Empty Drop | 2          | MyISAM        | utf8mb4_unicode_ci       | 2.1 KiB          | -            |
| employees        | ★ Browse Structure Search Insert Empty Drop | 8          | MyISAM        | utf8mb4_unicode_ci       | 2.6 KiB          | -            |
| gatepasss        | ★ Browse Structure Search Insert Empty Drop | 27         | MyISAM        | utf8mb4_unicode_ci       | 4.4 KiB          | 456 B        |
| gates            | ★ Browse Structure Search Insert Empty Drop | 2          | MyISAM        | utf8mb4_unicode_ci       | 2.1 KiB          | -            |
| invitations      | ★ Browse Structure Search Insert Empty Drop | 2          | MyISAM        | utf8mb4_unicode_ci       | 2.1 KiB          | -            |
| invitors         | ★ Browse Structure Search Insert Empty Drop | 1          | MyISAM        | utf8mb4_unicode_ci       | 2.0 KiB          | -            |
| migrations       | ★ Browse Structure Search Insert Empty Drop | 28         | MyISAM        | utf8mb4_unicode_ci       | 3.5 KiB          | -            |
| persistences     | ★ Browse Structure Search Insert Empty Drop | 54         | InnoDB        | utf8mb4_unicode_ci       | 32.0 KiB         | -            |
| requests         | ★ Browse Structure Search Insert Empty Drop | 4          | MyISAM        | utf8mb4_unicode_ci       | 2.2 KiB          | -            |
| roles            | ★ Browse Structure Search Insert Empty Drop | 5          | InnoDB        | utf8mb4_unicode_ci       | 32.0 KiB         | -            |
| role_users       | ★ Browse Structure Search Insert Empty Drop | 6          | InnoDB        | utf8mb4_unicode_ci       | 16.0 KiB         | -            |
| throttle         | ★ Browse Structure Search Insert Empty Drop | 49         | InnoDB        | utf8mb4_unicode_ci       | 32.0 KiB         | -            |
| users            | ★ Browse Structure Search Insert Empty Drop | 8          | InnoDB        | utf8mb4_unicode_ci       | 32.0 KiB         | -            |
| visitors         | ★ Browse Structure Search Insert Empty Drop | 3          | MyISAM        | utf8mb4_unicode_ci       | 3.5 KiB          | 232 B        |
| <b>16 tables</b> | <b>Sum</b>                                  | <b>736</b> | <b>MyISAM</b> | <b>latin1_swedish_ci</b> | <b>243.3 KiB</b> | <b>688 B</b> |

Figure 3 Database table

### 3.2.3 Use Case Diagram

A use case diagram is a behavioral UML diagram type and frequently used to analyze various systems. They enable you to visualize the different types of roles in a system and how those roles

interact with the system and also it is a way to summarize details of a system and the users within that system. It is generally shown as a graphic depiction of interactions among different elements in a system.

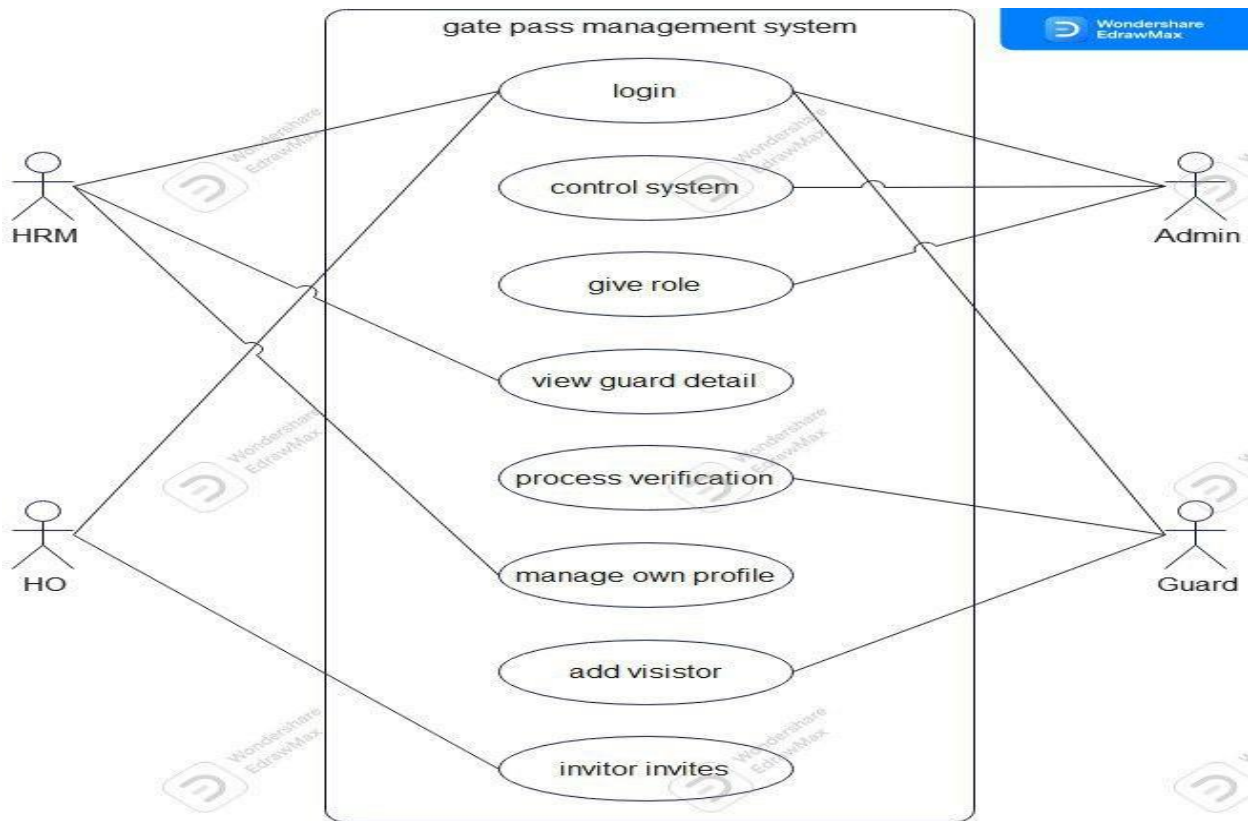


Figure 4 Use Case Diagram

### 3.2.4 Use Case Descriptions

Use case diagrams consist of actors, Actor in a use case diagram is any entity that performs a role in one given system. This could be a person, organization or an external system. In order to create relevant use cases for the gate pass management system, the following actors for the system have been identified:

- System admin
- Guard
- Manager(HR)
- HO(Inviter)

Use case descriptions included in Get pass management system are

- Log in
- Manage account
- View report
- View guard detail
- Restrict entry
- View entry gate detail
- Add visitor
- Invite visitor

### Use Case Description Using Table

|                     |                                                                                  |
|---------------------|----------------------------------------------------------------------------------|
| Use case name       | Manage account                                                                   |
| Participating actor | Admin                                                                            |
| Description         | Allows to provide functionalities to users; add, delete ,update and edit users . |
| Entry condition     | Admin must be login to the system to their own page                              |
| Exit condition      | Every user's account and system maintenance is managed as required.              |

*Table 1: use case description for manage account*

|                     |                                                                                                                       |
|---------------------|-----------------------------------------------------------------------------------------------------------------------|
| Use case name       | Login                                                                                                                 |
| Participating actor | Admin, Manager(HO),Guard                                                                                              |
| Description         | Any user who wants to access the system's functionality must be Authenticated and Authorized and login to the system. |
| Entry condition     | The user must be already registered (the user must have user name, password)                                          |
| Exit condition      | The system user logged in to the system.                                                                              |

*Table 2: use case description for login*

|                     |                   |
|---------------------|-------------------|
| Use case name       | View report       |
| Participating actor | Manager and guard |

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| Description     | This use case helps Every user to view out a report |
| Entry condition | Every user must be logged in to the system.         |
| Exit condition  | When reports are viewed by users                    |

*Table 3: use case description for view report*

|                     |                                                                              |
|---------------------|------------------------------------------------------------------------------|
| Use case name       | View guard detail                                                            |
| Participating actor | Manager(HO)                                                                  |
| Description         | Helps the manager(HO) to check or control guard profile (check guard status) |
| Entry condition     | The manager have to logged in                                                |
| Exit condition      | The required detail of guard viewed                                          |

*Table 4: use case description for view guard detail*

|                     |                                                                                                                                                                       |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use case name       | Restrict entry                                                                                                                                                        |
| Participating actor | Manager(HO) and guard                                                                                                                                                 |
| Description         | The HO and guard restrict (not allow) to get in the employee and visitor: if the employees are not registered in the system and for visitors if they are not invited. |
| Entry condition     | If unregistered or un invited user come                                                                                                                               |
| Exit condition      | Restrict the user                                                                                                                                                     |

*Table 5: use case description for restrict entry*

|                     |                                               |
|---------------------|-----------------------------------------------|
| Use case name       | View entry gate detail                        |
| Participating actor | Guard                                         |
| Description         | The guard view who entered and leave the gate |
| Entry condition     | The guard have to logged in                   |
| Exit condition      | View the entry/exit detail                    |

*Table 6: use case description for entry gate detail*

|                     |                                                                                                |
|---------------------|------------------------------------------------------------------------------------------------|
| Use case name       | Add visitor                                                                                    |
| Participating actor | Guard                                                                                          |
| Description         | If the visitor come the guard add the visitor into the visitor list and give them temporary id |
| Entry condition     | New visitor have to came                                                                       |
| Exit condition      | The visitor is added                                                                           |

*Table 7: use case description for Add visitor*

|                     |                                                                                          |
|---------------------|------------------------------------------------------------------------------------------|
| Use case name       | Invite visitor                                                                           |
| Participating actor | Inviter(HO)                                                                              |
| Description         | The HO add visitors to invitation list in order to check guard if the visitor is invited |
| Entry condition     | To invite visitor by logged in                                                           |
| Exit condition      | The visitor is invited                                                                   |

*Table 8: use case description for invite visitor*

### 3.2.5 Class Diagram

A class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.

Class diagram is a static diagram. It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.

Class diagram describes the attributes and operations of a class and also the constraints imposed on the system. The class diagrams are widely used in the modeling of object oriented systems because they are the only UML diagrams, which can be mapped directly with object-oriented languages.

Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

A class diagram is typically modeled rectangles with three-section:

- The top one indicates the name of the class
- The middle one lists the attributes of the class and
- The third one lists the methods

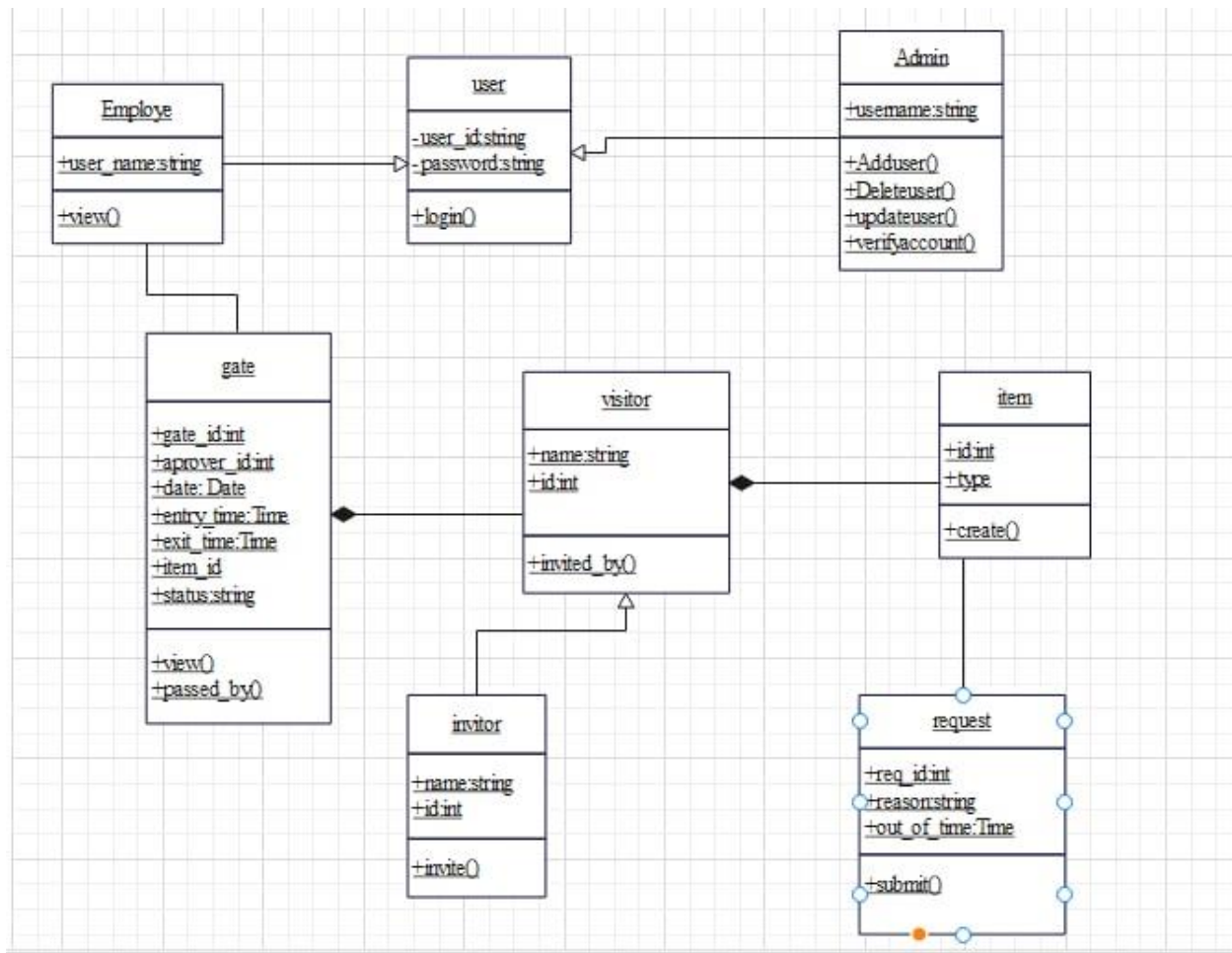


Figure 5 Class diagram

### 3.2.6 Sequence Diagram

A sequence diagram is a Unified Modeling Language (UML) diagram that illustrates the sequence of messages between objects in an interaction. A sequence diagram shows the sequence of messages passed between objects. Sequence diagrams can also show the control structures between objects. Sequence diagrams are a great way to validate and flesh out the logic of use case scenarios and to document the design of the system.

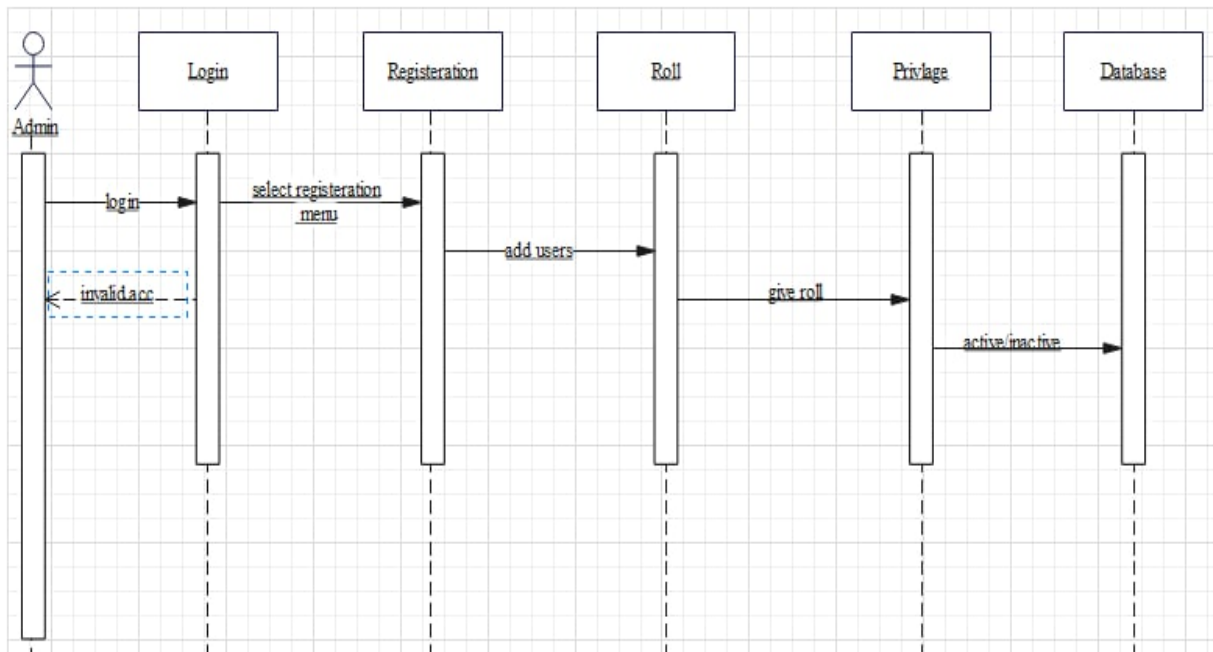


Figure 6: sequence diagram for admin

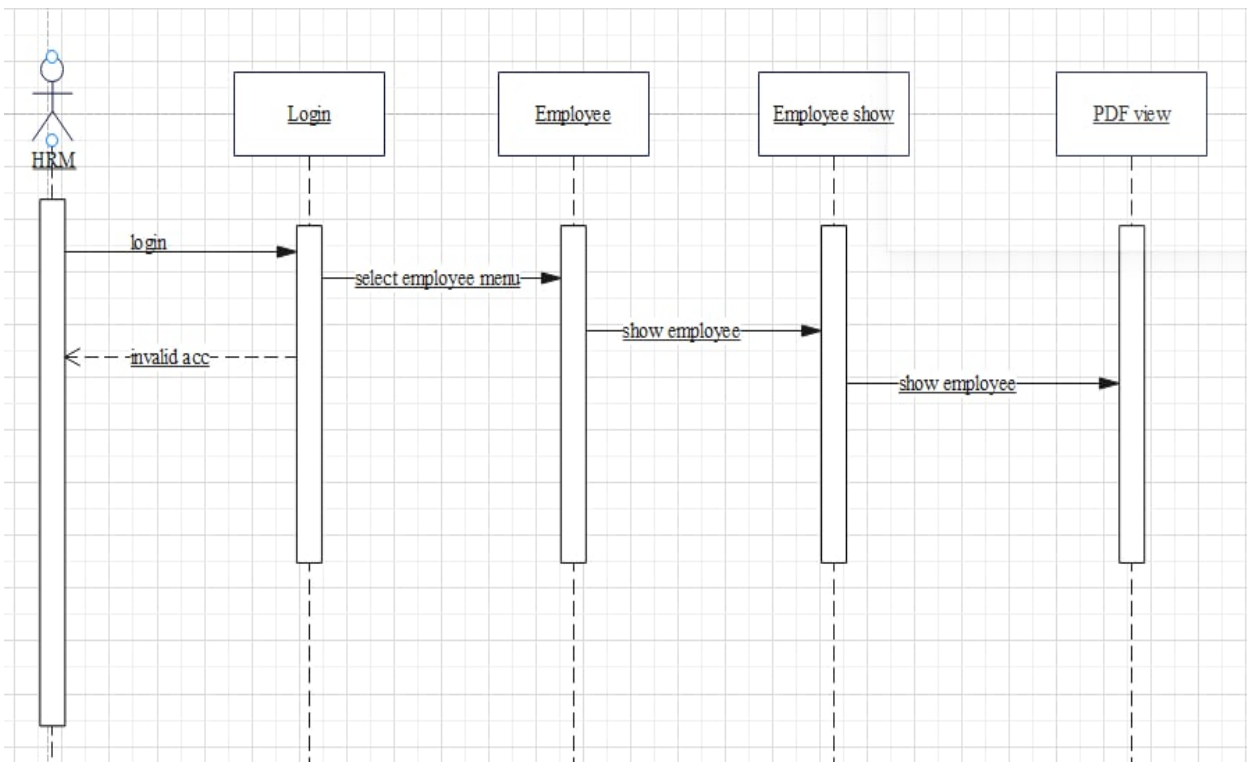


Figure 7 Sequence diagram for HRM generate id

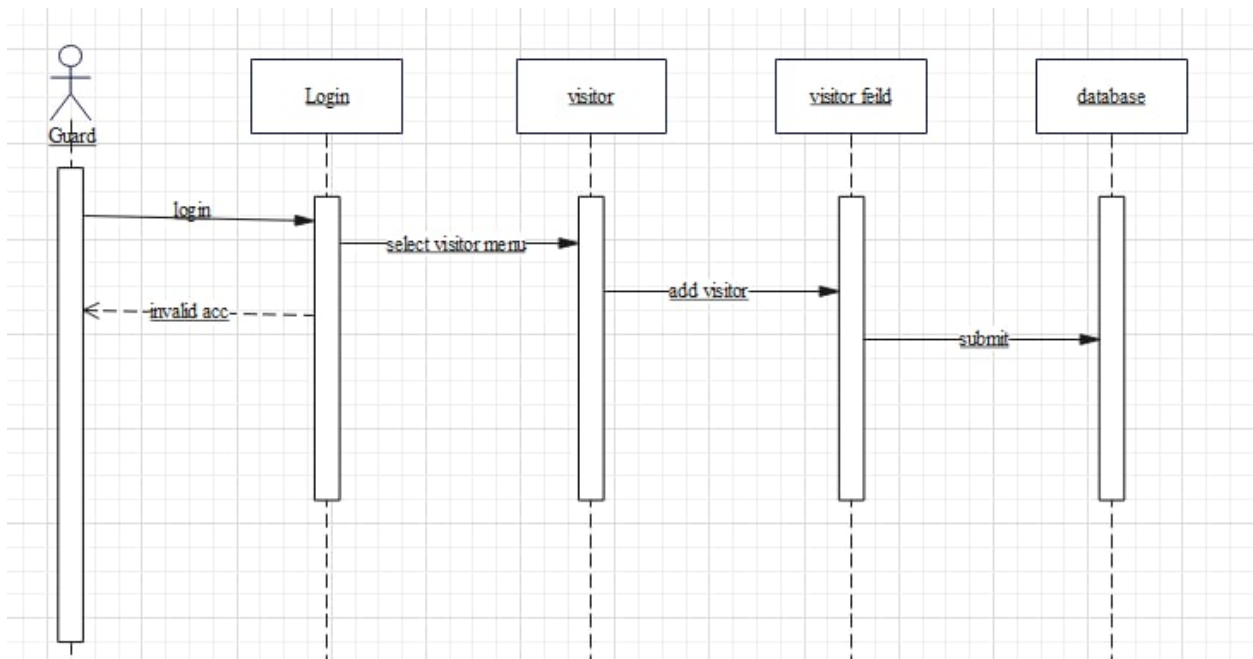


Figure 8: Sequence diagram for Guard add visitor

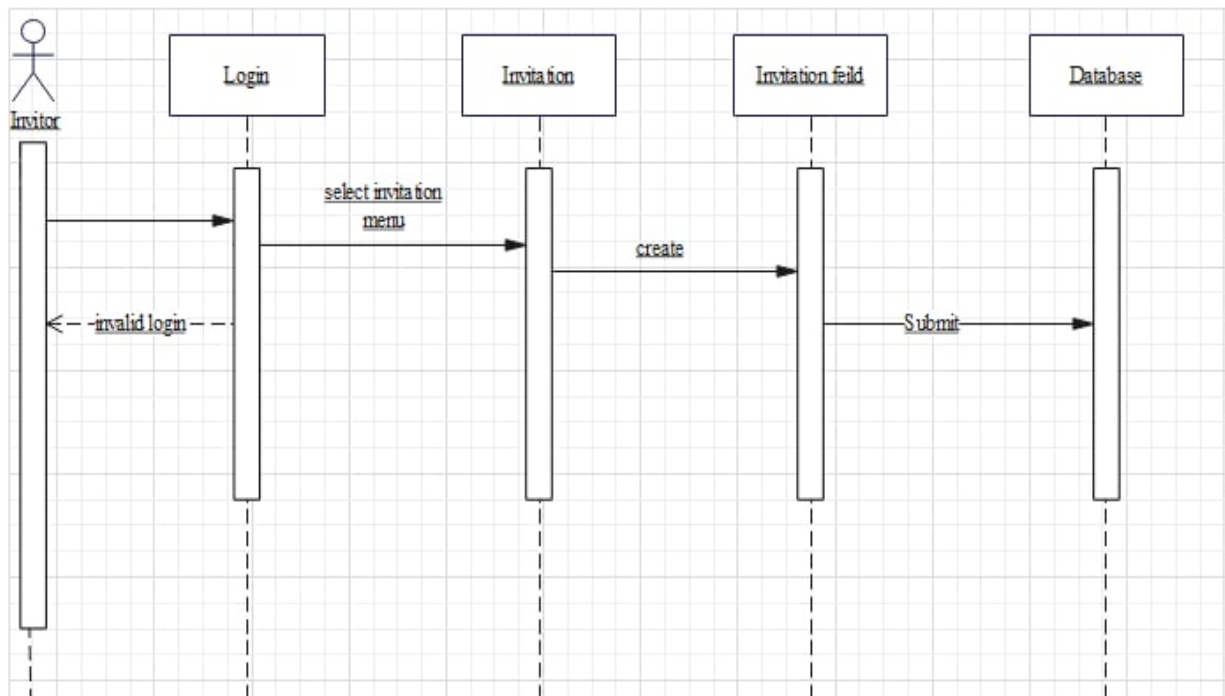


Figure 9: Sequence diagram for inviting invites visitor



## CHAPTER FOUR

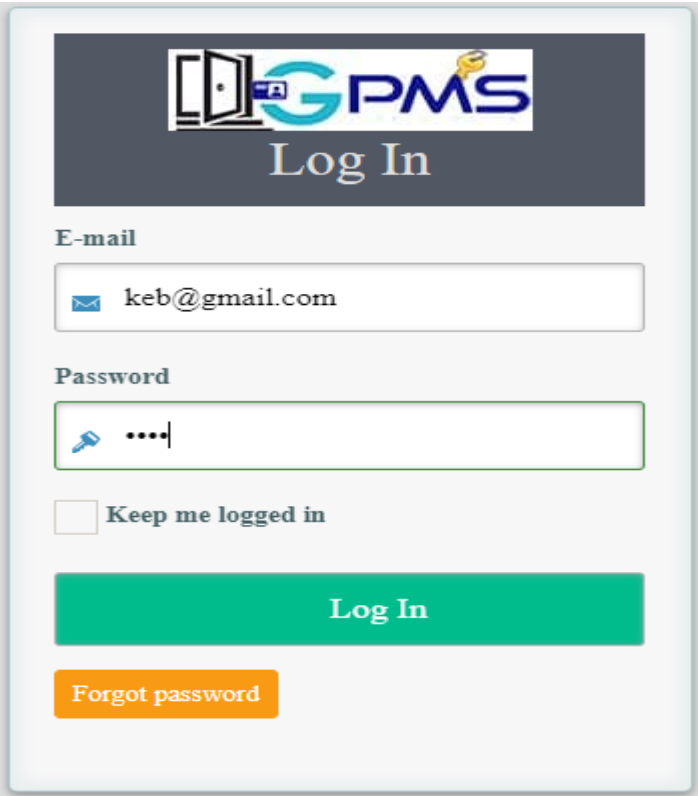
### RESULT AND DISCUSSION

The project's end product is a system that allows Company Gate Pass management system users to register and login to a system that is meant to monitor and keep employee and visitor gate records at the time of entry and exit from their respective premises. It's also utilized to create a digital identity card and track the total time spent entering and exiting using a web-based system. The project was completed using the larval framework.

In general, because it is a web-based application, it does not need a large infrastructure. Because our primary dealing with the Gate Pass management system is online, this system may be run simply by adding it to the main system.

The following figure shows login page which is used to authenticate users:

All users of the system by using their email address and password can signing to their account

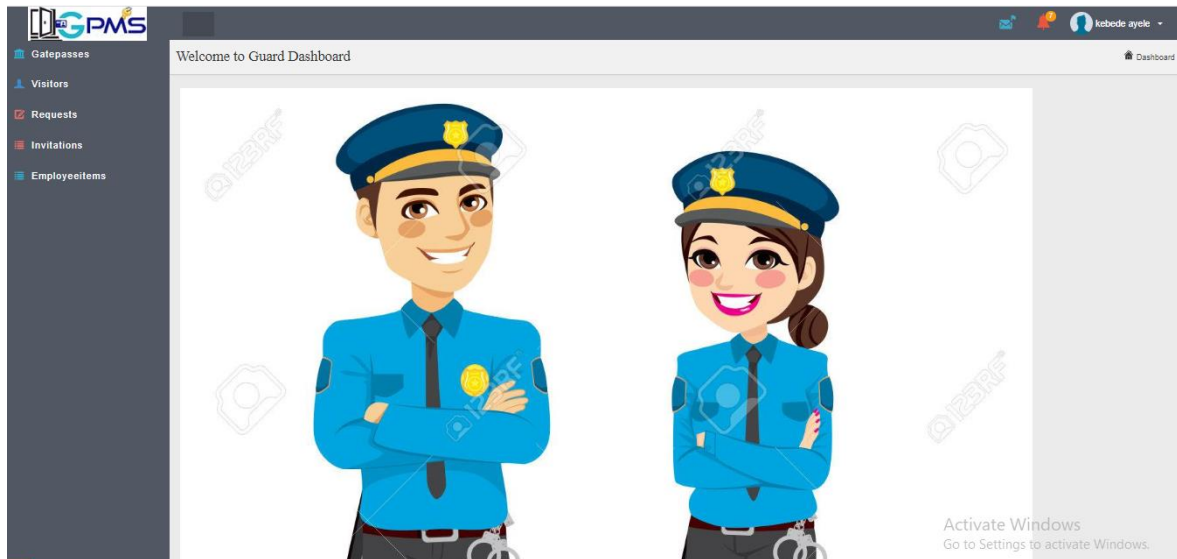


The screenshot displays the login interface for the GPMS (Gate Pass Management System). At the top, there is a header with a logo consisting of a door icon and the text 'GPMS', followed by 'Log In' in a large, serif font. Below the header, there are two input fields: 'E-mail' and 'Password'. The 'E-mail' field contains the text 'keb@gmail.com'. The 'Password' field contains a key icon and four dots, indicating a masked password. Below these fields, there is a checkbox labeled 'Keep me logged in'. At the bottom, there is a large green button labeled 'Log In' and a smaller orange button labeled 'Forgot password'.

*Figure 10: Login Page*

**The following figure shows home page for Guard**

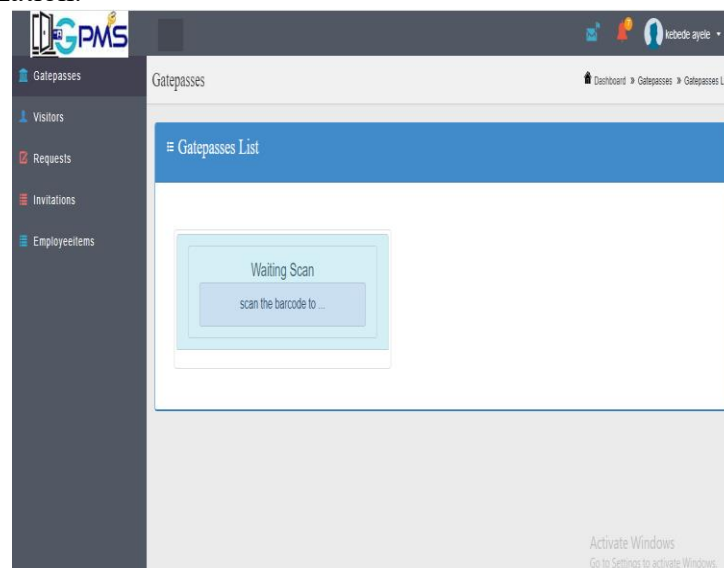
After login the guard is able to access his/her account and able to do tasks that are given by the system admin including being able to change the password of his/her account.



*Figure 11: Guard Home Page*

**The following figure shows how the barcode reader display**

Employees and visitors show their id to the barcode reader at the gate and the guard after waiting some time until the system verifies the success or not valid message allow or not allow them to gate into the organization.



*Figure 12: Waiting to Scan*

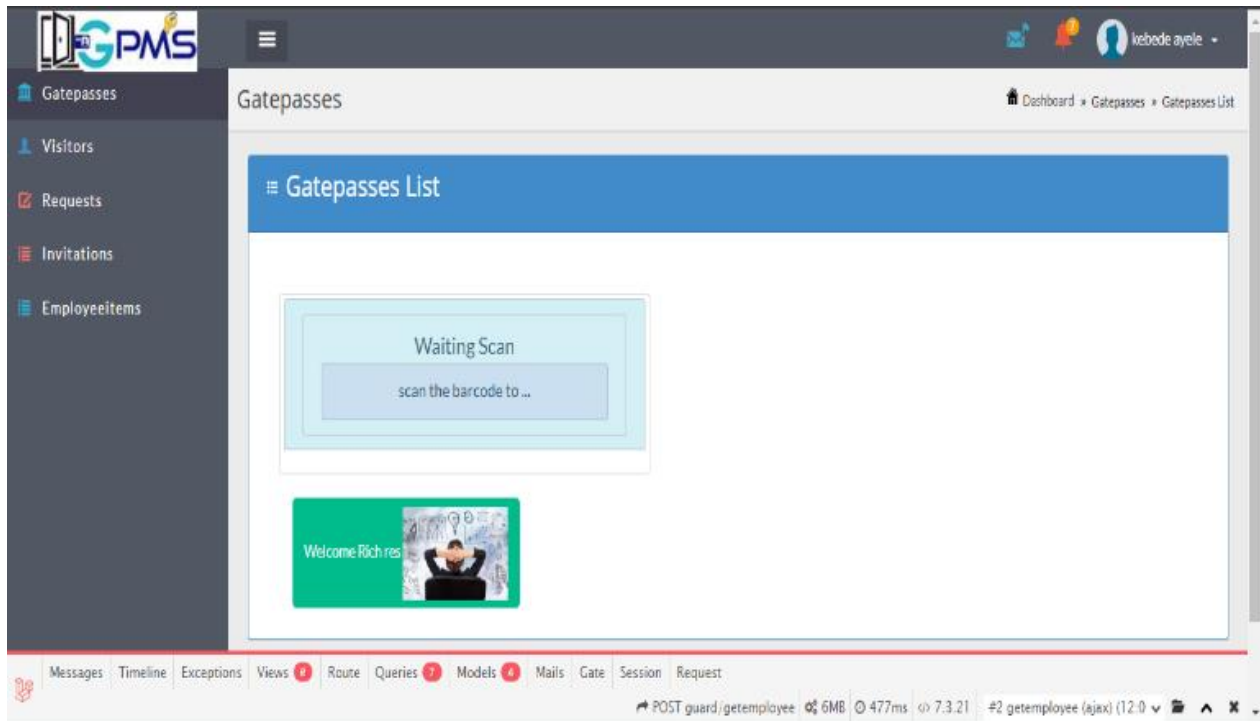


Figure 13: Barcode When an employee enters

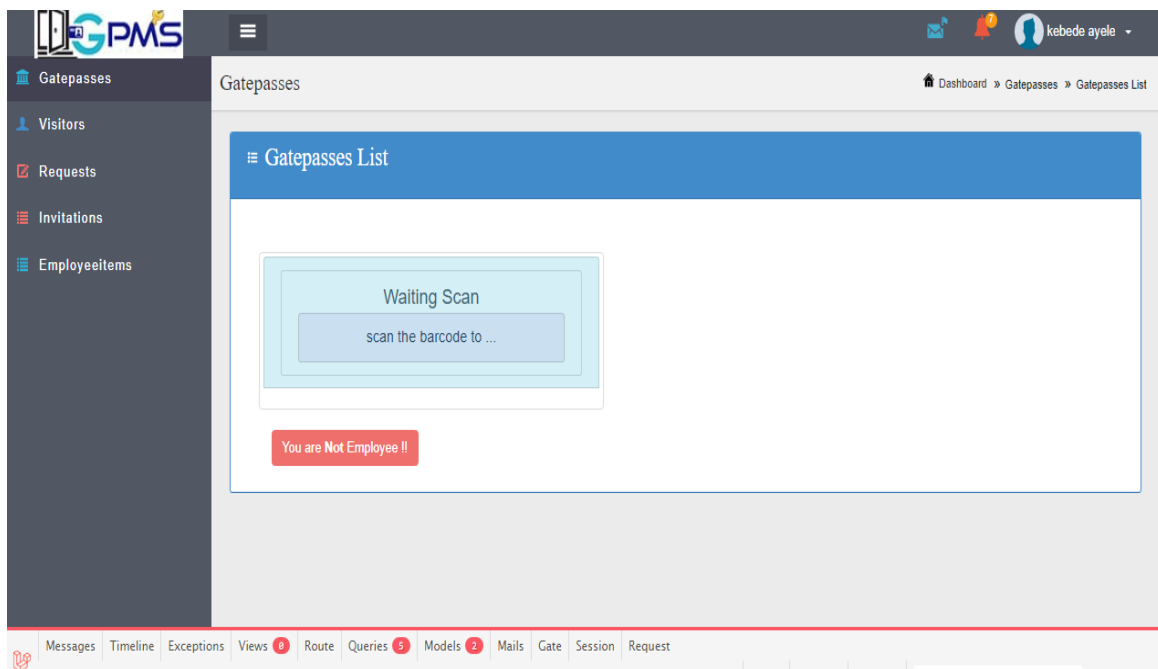


Figure 14: Barcode reader when it is not employee

The following figure shows how the Guard add visitor and give them temporary id

When visitor came to the organization for different purpose the guard prepare a temporary ID for Visitor in order to make it proper and handle the information of the visitor in the system.

**Edit Visitor**

First Name:

Middle Name:

Last Name:

Gender:

Phone:

Address:

Person Title:

Reason:

Visitor Id:

Waiting Time:

Item:

Serial:

Figure 15: Guard adds visitor

The following figure shows the Guard view request and visitor list.

When the higher official prepares some event and invites visitors for different purposes, send it to the guard using the system. And the guard sees the details of it.

**Invitations List**

Show 10 entries

| Event    | Person Name | Date       | Action                              |
|----------|-------------|------------|-------------------------------------|
| training | ss          | 12/05/2022 | <input type="button" value="show"/> |

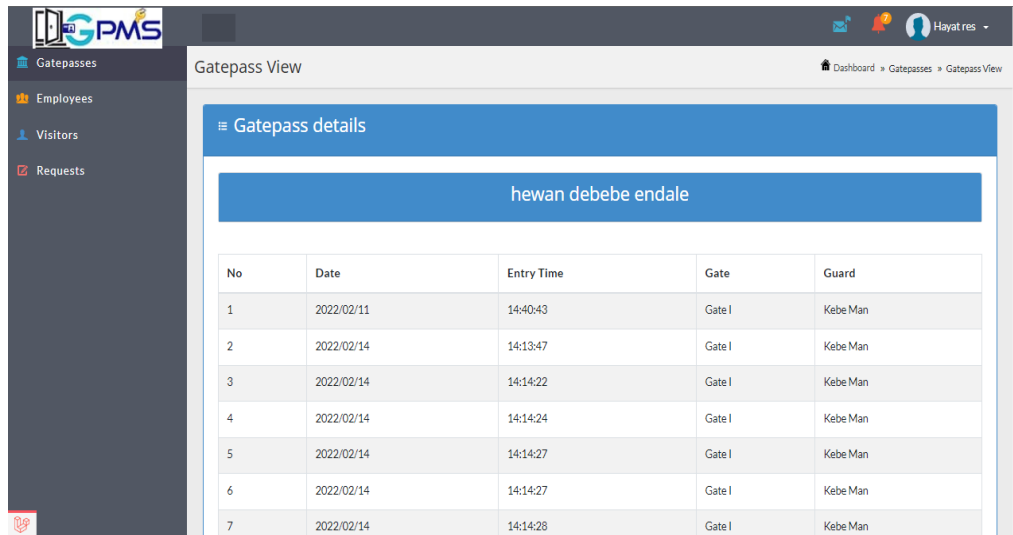
Showing 1 to 1 of 1 entries

Previous 1 Next

*Figure 16: Guard request view page*

The following figure shows how the HRM view gate passes report

The human resource manager fills the details of the employee in the organization



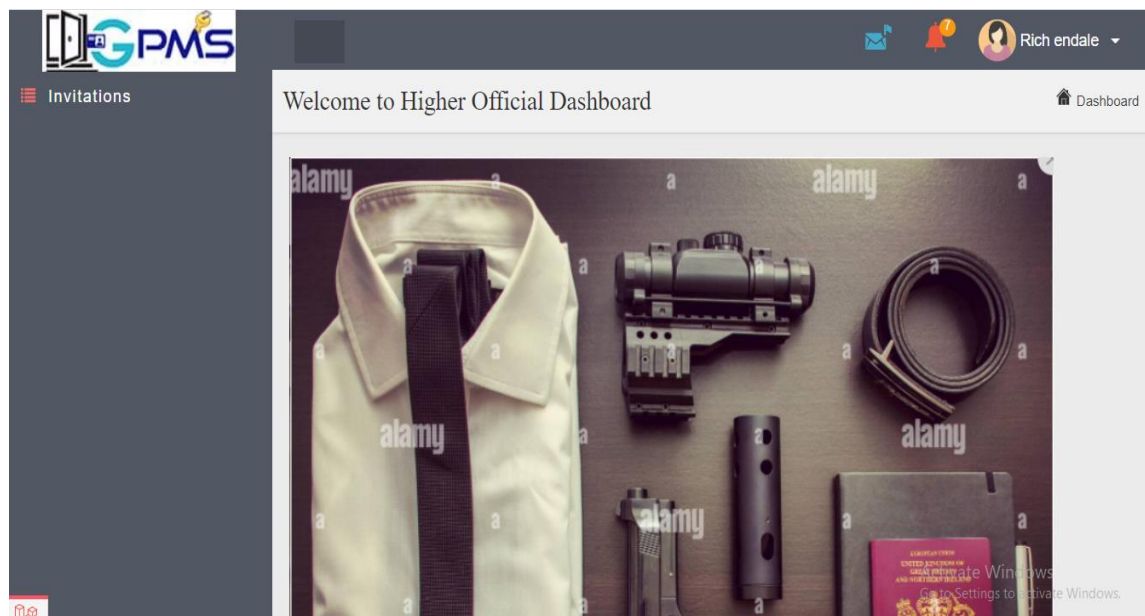
The screenshot shows the 'Gatepass View' page in the GPMs system. The page title is 'Gatepass View' and the user is 'Hayat res'. The left sidebar contains links for Gatepasses, Employees, Visitors, and Requests. The main content area is titled 'Gatepass details' and shows the name 'hewan debebe endale'. Below this is a table with the following data:

| No | Date       | Entry Time | Gate   | Guard    |
|----|------------|------------|--------|----------|
| 1  | 2022/02/11 | 14:40:43   | Gate I | Kebe Man |
| 2  | 2022/02/14 | 14:13:47   | Gate I | Kebe Man |
| 3  | 2022/02/14 | 14:14:22   | Gate I | Kebe Man |
| 4  | 2022/02/14 | 14:14:24   | Gate I | Kebe Man |
| 5  | 2022/02/14 | 14:14:27   | Gate I | Kebe Man |
| 6  | 2022/02/14 | 14:14:27   | Gate I | Kebe Man |
| 7  | 2022/02/14 | 14:14:28   | Gate I | Kebe Man |

*Figure 17: HRM report view page*

The following figure shows how the HO (inviter) create invitation

Higher officials create invitations and send full information to the guard after filling the details of the visitor .

*Figure 18: HO home Page*

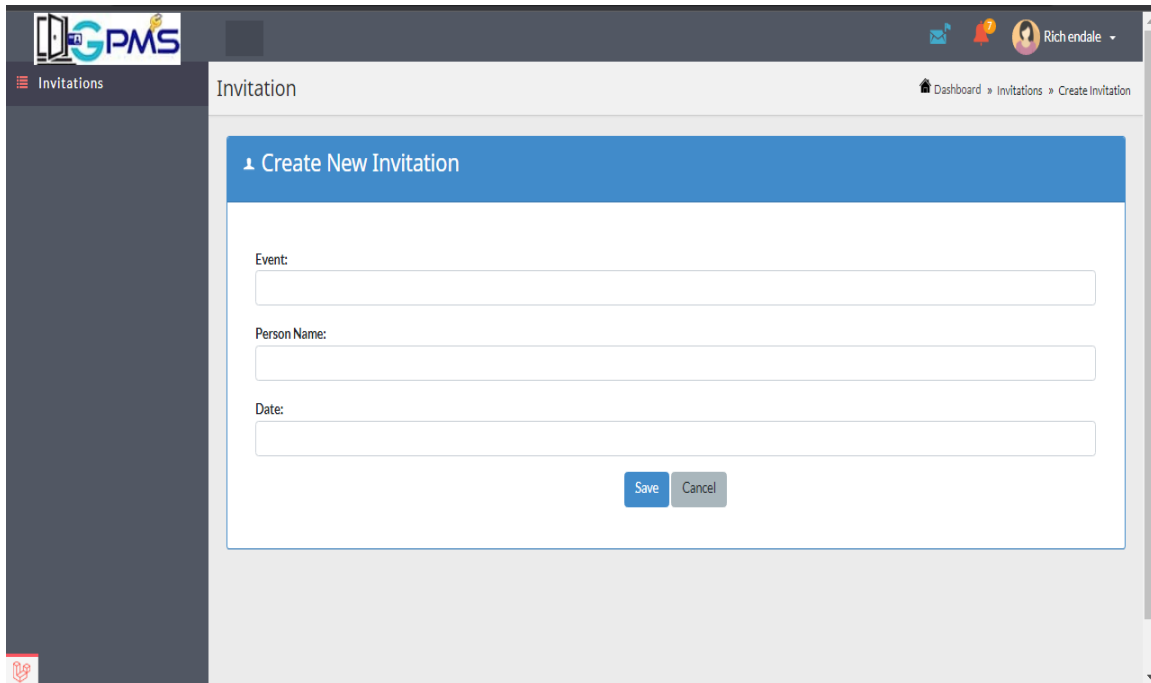


Figure 19: HO create invitation

**The following figure shows home page for Admin**

The system administration controls the overall system and gives a role for the users in the system.

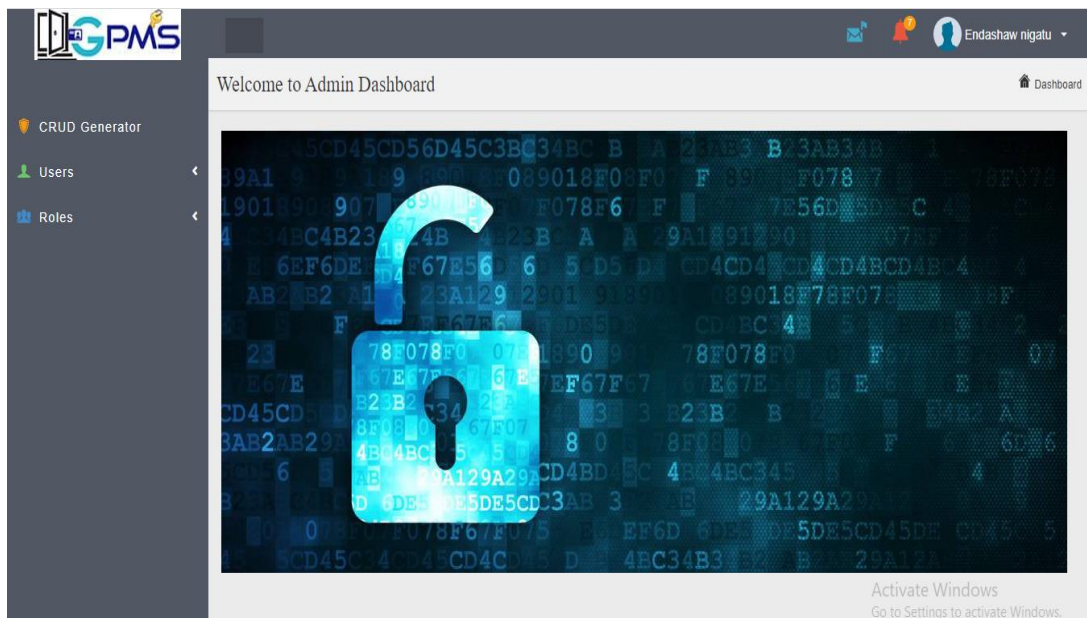
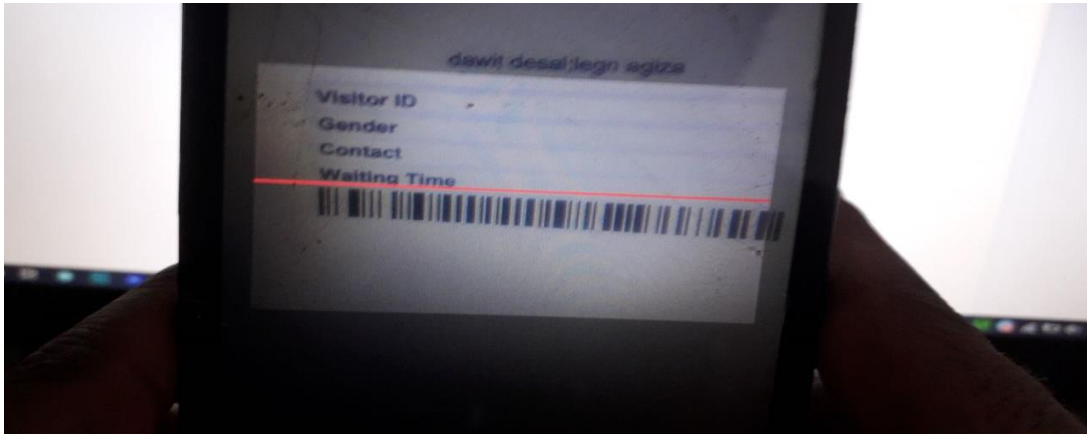


Figure 20: Admin Home Page

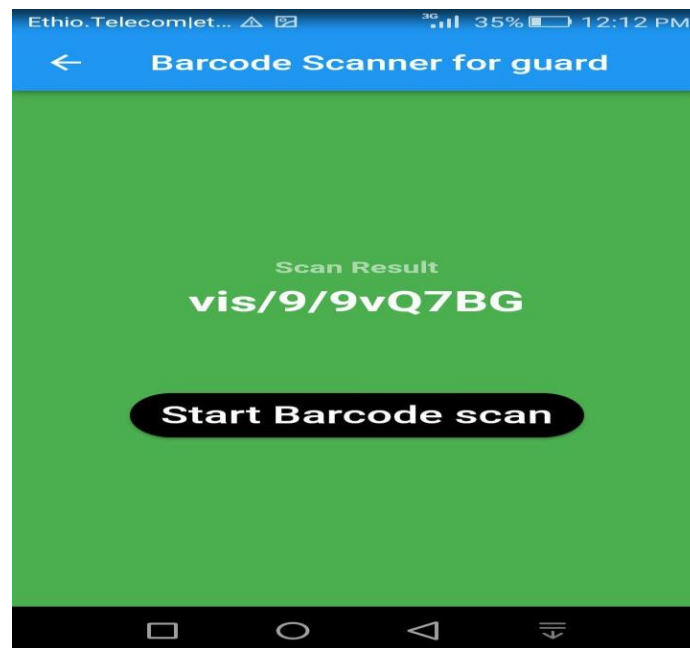
**The following show the scanning process of the android application**

Mobile application after scanning the visitor and employee id generated by the system sends the scanned result to the website database using the post method and receives the verification from the website database to the application using the get method.



*Figure 21: scanning using app*

After the application finishes the scanning process it will display the result like below.



*Figure 22: result of scanning the barcode*

### **Integrating mobile application with website by using API**

API stands for an application programming interface. Lately there has been a lot of conversation regarding the benefits of API for developers.

APIs are the set of rules which define how mobile apps exchange data with other apps. API's help to accept and return the information as per the application requirements.

When access is provided to an API, the content generated can be published automatically and is available for every channel. It allows it to be shared and distributed more easily. Integration: APIs allow content to be embedded from any site or application more easily..



## CHAPTER FIVE

### CONCLUSION AND RECOMMENDATION

#### 5.1 Conclusion

Based on the record in the system analysis stage, the manual pass slip's current technical system was time-consuming for recording the file, inconsistency, and difficulty generating reports. Automated Gate Pass Management System is then believed to be a very convenient way to store, manipulate, analyze, retrieve, and track the institution's information on gate pass. It can generate real-time data as needed and adaptable to new or modified information requirements. Thus, it is more practical and convenient for the system to be supported on its full implementation. Lastly, the system conforms to usability and functionality standards, which provides ease in decision-making among top management.

#### 5.2 Recommendation

The scope of our project is limited to the security and reporting of gate passes for entrance purposes only; however, we recommend that you work on an in and out (entry/exit) gate pass management system for employees that will be used to track employee attendance as they enter and exit.

## Reference

- [1] "About ASTU:Background of ASTU," 2020. [Online]. Available: <https://www.astu.edu.et/>. [Accessed 8 February 2022].
- [2] "About ASTU:Mission and Purpose," 2020. [Online]. Available: <https://www.astu.edu.et/>. [Accessed 8 February 2022].
- [3] P. kamanu, "VISITOR AND GATE PASS MANAGEMENT SYSTEM," MULTIMEDIA UNIVERSITY OF KENYA, Kenya, 2020. [4] "tutorials point," [Online]. Available: <https://www.tutorialspoint.com/>. [Accessed 20 February 2022].

## Appendix

### Gate pass controller for guard

```
<?php

namespace App\Http\Controllers\Guard;

use App\Http\Requests;

use App\Http\Requests\CreateGatepassRequest;

use App\Http\Requests\UpdateGatepassRequest;

use App\Repositories\GatepassRepository;

use App\Http\Controllers\AppBaseController as InfyOmBaseController;

use Illuminate\Http\Request;

use Illuminate\Support\Facades\Lang;

use App\Models\Gatepass;

use App\Models\Employee;

use App\Models\Gate;

use Flash;

use Prettus\Repository\Criteria\RequestCriteria;

use Response;

use Auth;

use Sentinel;
```

```
class GatepassController extends InfyOmBaseController

{

    /** @var Gate pass Repository */

    private $gatepassRepository;

    public function __construct(GatepassRepository $gatepassRepo)

    {

        $this->gatepassRepository = $gatepassRepo;

    }

    /**

    * Display a listing of the Gate Pass.

    *

    * @param Request $request

    * @return Response

    */

    public function index(Request $request)

    {

        $this->gatepassRepository->pushCriteria(new RequestCriteria($request));

        $gatepasses = $this->gatepassRepository->all();

        return view('guard.gatepasses.index')
    }
}
```

```
->with('gatepasses', $gatepasses);

}

/**

 * Show the form for creating a new Gate Pass.

 *

 * @return Response

 */

public function create()

{

    return view('guard.gatepasses.create');

}

/**

 * Store a newly created Gate Pass in storage.

 *

 * @param CreateGatepassRequest $request

 *

 * @return Response

 */

public function store(CreateGatepassRequest $request)
```

```
{  
  
    $request->request->add(['approved_by' => $request->user()->id]);  
  
    $input = $request->all();  
  
    $gate pass = $this->gatepassRepository->create($input);  
  
    Flash::success('Gatepass saved successfully.');
```

```
    return redirect(route('guard.gatepasses.index'));
```

```
}
```

```
/**
```

```
 * Display the specified Gate Pass.
```

```
 *
```

```
 * @param int $id
```

```
 *
```

```
 * @return Response
```

```
 */
```

```
public function show($id)
```

```
{
```

```
    $gate pass = $this->gatepassRepository->findWithoutFail($id);
```

```
    if (empty($gate pass)) {
```

```
        Flash::error('Gate Pass not found');
```

```
        return redirect(route('gatepasses.index'));

    }

    return view('guard.gatepasses.show')->with('gate pass', $gate pass);

}

/**

 * Show the form for editing the specified Gate Pass.

 *

 * @param int $id

 *

 * @return Response

 */

public function edit($id)

{

    $gate pass = $this->gatepassRepository->findWithoutFail($id);

    if (empty($gate pass)) {

        Flash::error('Gate Pass not found');

        return redirect(route('gatepasses.index'));

    }

    return view('guard.gatepasses.edit')->with('gate pass', $gate pass);
```

```
}

/**
 * Update the specified Gate Pass in storage.
 *
 * @param int      $id
 * @param UpdateGatepassRequest $request
 *
 * @return Response
 */

public function update($id, UpdateGatepassRequest $request)
{
    $gate pass = $this->gatepassRepository->findWithoutFail($id);

    if (empty($gate pass)) {
        Flash::error('Gate Pass not found');

        return redirect(route('gatepasses.index'));
    }

    $gate pass = $this->gatepassRepository->update($request->all(), $id);

    Flash::success('Gate Pass updated successfully. ');

    return redirect(route('guard.gatepasses.index'));
```



```
}

/**
 * Remove the specified Gate Pass from storage.
 *
 * @param int $id
 *
 * @return Response
 */

public function getModalDelete($id = null)
{
    $error = "";

    $model = "";

    $confirm_route = route('guard.gatepasses.delete',['id'=>$id]);

    return View('guard.layouts/modal_confirmation', compact('error','model',
'confirm_route'));
}

public function getDelete($id = null)
{
    $sample = Gate Pass::destroy($id);

    // Redirect to the group management page
```

```
        return redirect(route('guard.gatepasses.index'))->with('success',
Lang::get('message.success.delete'));

    }

    // For Barcode

    public function barcode reader(Request $request)

    {

        $employee = Employee::where('id_number',$request->barcode)->first();

        $date = date('Y/m/d H:i:s');

        $today = explode(' ', $date);

        $result = "";

        if (empty($employee)) {

            $result = "<span class='btn btn-danger'>You are <b>Not</b> Employee
!!</span>";

        }else{

            $result = "<span class='btn btn-success'> Welcome ".$employee->first_name."
".$employee->middel_name." ".$employee->last_name." <img
src='http://gpms.local:8089/employees/profile/".$employee->photo.'" height='80px'
alt='image'/></span>";

            $logged_user = Sentinel::getUser()->id;

            $gate_id = Sentinel::getUser()->gate_id;

            $gate = Gate::where('id', $gate_id)->first();
```

```
$gate pass = new Gate Pass([  
  
    'gate_id' => $gate->g__name,  
  
    'entry_time' => $today[1],  
  
    'emp_id'=> $employee->id,  
  
    'date' => $today[0],  
  
    'purpose_of_visit' => 'Work',  
  
    'state' => 'default',  
  
    'status' => 'accepted',  
  
    'guard_id' => $logged_user  
  
]);  
  
$gate pass->save();  
  
}  
  
return response()->json(['employee'=>$result]);  
  
}  
  
}
```

**application barcode scanner**

```
import 'package:barcode_scanner_guard/main.dart';

import 'package:barcode_scanner_guard/widget/button_widget.dart';

import 'package:flutter/material.dart';

import 'package:flutter/services.dart';

import 'package:flutter_barcode_scanner/flutter_barcode_scanner.dart';

class BarcodeScanPage extends StatefulWidget {

  @override

  _BarcodeScanPageState createState() => _BarcodeScanPageState();

}

class _BarcodeScanPageState extends State<BarcodeScanPage> {

  String barcode = 'Unknown';

  @override

  Widget build(BuildContext context) => Scaffold(

    appBar: AppBar(

      title: Text(MyApp.title),

    ),

    body: Center(

      child: Column(
```

*mainAxisAlignment: MainAxisAlignment.center,*

*children: <Widget>[*

*Text(*

*'Scan Result',*

*style: TextStyle(*

*fontSize: 16,*

*color: Colors.white54,*

*fontWeight: FontWeight.bold,*

*),*

*),*

*SizedBox(height: 8),*

*Text(*

*'\$barcode',*

*style: TextStyle(*

*fontSize: 28,*

*fontWeight: FontWeight.bold,*

*color: Colors.white,*

*),*

*),*

```
        SizedBox(height: 72),

        ButtonWidget(

          text: 'Verify',

          onClicked: mainDart,

        ),

        SizedBox(height: 72),

        ButtonWidget(

          text: 'Start Barcode scan',

          onClicked: scanBarcode,

        ),

      ],

    ),

  );

  Future<void> scanBarcode() async {

    try {

      final barcode = await FlutterBarcodeScanner.scanBarcode(

        '#ff6666',

        'Cancel',
```

```
    true,

    ScanMode.BARCODE,

  );

  if (!mounted) return;

  setState() {

    this.barcode = barcode;

  });

} on PlatformException {

  barcode = 'Failed to get platform version.';

}

}

Future<void> mainDart() async {

try {

  final result = await FlutterBarcodeScanner.scanBarcode(

    '#ff6666',

    'Cancel',

    true,

    ScanMode.BARCODE,);

  if (!mounted) return;

  setState() {
```

```
this.result= result;  
  
});  
  
} on PlatformException {  
  
barcode = 'Failed to get platform version.';  
  
}  
  
}  
  
}
```